IsoVerify: SMT-Based Verification of Transaction Isolation Guarantees with Witness Validation

halley-labs

2026

Abstract

Database systems promise specific transaction isolation levels—serializable, snapshot isolation, read committed—but subtle implementation bugs can violate these guarantees. Existing testing tools like Jepsen and Elle detect anomalies through black-box testing but provide no formal proofs and can miss rare violations. We present IsoVerify, a verification tool that builds dependency graphs from transaction logs, checks for forbidden cycle patterns using Adya's formalism, and produces machine-checkable certificates. A key contribution is *counterexample-guided witness validation*: every candidate anomaly is confirmed by a concrete witness (specific transactions and operations) before being reported, eliminating the false positives that plagued earlier SMT encodings. On the main benchmark of 33 synthetic transaction histories, IsoVerify and a simplified Elle-style detector both achieve 3.0% accuracy, so the current benchmark does not provide evidence of a meaningful comparative advantage for either approach. On a separate 18-case coverage suite of tiny, targeted histories, IsoVerify detects all embedded anomaly patterns after witness validation; we treat those results as functionality checks rather than evidence of real-world performance. *Important caveats:* all benchmark histories are synthetic, the Elle baseline is a simplified reimplementation rather than the published tool, ground-truth labels are generated by the same codebase that performs the evaluation, and some earlier headline numbers came from an unreproducible static results file (Section 10).

1 Introduction

Transaction isolation is a cornerstone of database correctness. The SQL standard defines four isolation levels (read uncommitted, read committed, repeatable read, serializable), and many systems offer additional levels like snapshot isolation (SI) and serializable snapshot isolation (SSI). However, the gap between *promised* and *delivered* isolation is well-documented:

- Kingsbury's Jepsen project [1] found isolation violations in CockroachDB, YugabyteDB, TiDB, and other production databases.
- Biswas and Enea [4] showed that the complexity of checking transactional consistency is NP-hard for many isolation levels.
- Tan et al.'s Cobra [3] demonstrated that polynomial-time verification is possible for snapshot isolation specifically.

Current tools face fundamental limitations:

1. **Incompleteness**: Black-box testing (Jepsen [1], Elle [2]) can detect anomalies it encounters but cannot prove their absence—missing an anomaly doesn't mean the system is correct.

2. **Informality**: Test verdicts are not machine-checkable proofs; they rely on tool correctness.
3. **Anomaly coverage**: Most tools check for specific anomaly patterns (e.g., G2-item) but not the full anomaly taxonomy.

IsoVerify provides formal verification by:

1. Constructing complete dependency graphs from transaction traces (Section 3).
2. Checking for forbidden cycles using Adya et al.'s [8] formal isolation definitions (Section 4).
3. Producing machine-checkable certificates of compliance or violation (Section 6).
4. Validating every candidate anomaly with a concrete witness before reporting it (Section 5).
5. Supporting all standard isolation levels plus SI and SSI (Section 7).

2 Related Work

Black-Box Isolation Testing. Jepsen [1] (Kingsbury) is the de facto standard for distributed systems testing, including isolation level verification. It generates random transactions, observes results, and checks for anomalies. Elle [2] (Kingsbury & Alvaro, VLDB 2020) improves on Jepsen with Adya-based anomaly checking over list-append histories. Both are powerful testing tools but fundamentally incomplete: passing all tests does not guarantee correctness.

Formal Isolation Checking. Cobra [3] (Tan et al., OSDI 2020) checks snapshot isolation using polynomial-time algorithms for specific anomaly patterns. It achieves good performance but only verifies SI, not serializable or other levels. PolySI [7] (Huang et al., PVLDB 2023) provides efficient black-box SI checking with completeness guarantees. Biswas and Enea [4] (OOPSLA 2019) established the complexity landscape for checking transactional consistency across isolation levels.

Predictive Analysis. IsoPredict [5] (Geng et al., PLDI 2024) uses dynamic predictive analysis to detect unserializable behaviors in weakly isolated data store applications. MonkeyDB [6] (Biswas et al., SIGMOD 2021) injects weak isolation behaviors to test application correctness. These tools complement verification by generating violation-revealing workloads.

Formal Foundations. Adya's thesis [8] provides the definitive formalization of isolation levels using dependency graphs and forbidden cycle patterns. Cerone et al. [9] extended this to distributed databases. Crooks et al. [10] formalized client-centric isolation. IsoVerify builds directly on Adya's formalism with certificate generation.

3 Transaction Dependency Graphs

Definition 1 (Dependency Graph)**.** *For a transaction history H, the dependency graph $G_H = (V, E)$ has transactions as vertices and edges labeled by dependency type:*

- $T_i \xrightarrow{ww} T_j$*: write-write dependency (T_j overwrites T_i's write).*
- $T_i \xrightarrow{wr} T_j$*: write-read dependency (T_j reads T_i's write).*

```
Input: History H, target isolation level I
Output: Verification result with certificate
G_H ← BuildDependencyGraph(H);
cycles ← FindForbiddenCycles(G_H, I);
if |cycles| = 0 then
    π ← GenerateComplianceCert(G_H, I);
    return (COMPLIANT, π);
end
else
    anomaly ← ClassifyCycle(cycles[0]);
    π ← GenerateViolationCert(anomaly, G_H);
    return (VIOLATION, anomaly, π);
end
```

Algorithm 1: IsoVerify Verification Algorithm

- $T_i \xrightarrow{rw} T_j$: *read-write (anti-dependency, T_j overwrites a value T_i read).*

Theorem 2 (Graph Construction Soundness and Completeness)**.** *For any transaction history H with read-from and version-order information, IsoVerify constructs a dependency graph G_H that is* complete*: every dependency derivable from H appears in G_H, and* sound*: every edge in G_H corresponds to an actual dependency.*

Proof sketch. See Appendix A.

IsoVerify extracts dependencies from SQL traces by:

1. Parsing transaction boundaries (BEGIN/COMMIT/ROLLBACK).
2. Tracking read-from relations via value matching or explicit version stamps.
3. Inferring version order from write timestamps or commit order.
4. Constructing anti-dependencies from read sets and subsequent writes.

4 Isolation Level Verification

Following Adya's formalism, each isolation level is defined by its forbidden cycle patterns:

Definition 3 (Isolation Level Characterization)**.**

- ***Read Uncommitted****: Forbids G0 (write cycles: cycles with only ww edges).*
- ***Read Committed****: Forbids G0 + G1a (aborted reads) + G1b (intermediate reads) + G1c (circular information flow with wr edges).*
- ***Repeatable Read****: Forbids G0 + G1 + G2-item (anti-dependency cycles on individual items).*
- ***Snapshot Isolation****: Forbids G0 + G1 + G-SIa (write skew with specific structure).*
- ***Serializable****: Forbids all cycles in the dependency graph.*

Theorem 4 (Verification Soundness and Completeness). *For a complete dependency graph G_H, IsoVerify's cycle detection is both sound and complete: it reports a violation if and only if the history H violates isolation level I according to Adya's formalism.*

Proof sketch. See Appendix B.

Theorem 5 (Verification Complexity). *For a history with n transactions and m operations:*

- *Graph construction: $O(m \log m)$ (sorting operations + hash-based dependency detection).*
- *Serializable checking: $O(n + |E|)$ (topological sort / cycle detection).*
- *SI checking: $O(n^2 \cdot |E|)$ (checking write-skew cycle structure).*
- *Certificate generation: $O(|E|)$ (linear in graph size).*

Proof sketch. See Appendix C.

5 Counterexample-Guided Witness Validation

A naïve SMT encoding may report anomalies that do not correspond to actual isolation violations. For example, a timing-based dirty-read check can fire whenever an operation timestamp precedes a commit timestamp, even if the read observes a properly committed value (Section 9). To eliminate such false positives, IsoVerify applies a *witness validation* pass: after the checker produces a set of candidate anomalies, each candidate is accepted only if a concrete witness—specific transactions and operations—substantiates it. This is analogous to counterexample-guided abstraction refinement (CEGAR): the initial analysis over-approximates, and the validation pass removes spurious reports.

Witness requirements per anomaly type:

- **Dirty read (G1a):** A committed transaction T_r reads a version v of key k that was written by an aborted transaction T_w.
- **Lost update:** Two committed transactions T_1, T_2 both read the same version of key k and then both write k.
- **Write skew (G-SIa):** Two transactions read overlapping key sets and write disjoint key sets.
- **Phantom read (G2):** A single transaction reads an aggregate/predicate key at two different versions.
- **Read skew:** A single transaction reads two keys whose versions come from inconsistent snapshots (one pre- and one post- an intervening committed write).

Only candidates with a valid witness are reported.

6 Machine-Checkable Certificates

Definition 6 (Compliance Certificate). *A compliance certificate $\mathcal{C}_{ok} = (G_H, I, \sigma)$ consists of the dependency graph, the isolation level, and a topological ordering σ of G_H (for serializable) or a valid commit timestamp assignment (for SI) witnessing compliance.*

Definition 7 (Violation Certificate). *A violation certificate $\mathcal{C}_{viol} = (G_H, I, \gamma, type)$ consists of the dependency graph, isolation level, a forbidden cycle γ, and its anomaly classification (dirty read, non-repeatable read, phantom, write skew, lost update).*

Certificate verification is $O(n)$: check that σ is a valid topological order (compliance) or that γ forms a forbidden cycle (violation).

Table 1: Benchmark suite composition. *Note:* all histories are synthetically generated by the benchmark scripts, not derived from real database logs.

Category	Count	Txns	Anomaly types
Small	13	2	dirty read, lost update, write skew, phantom, read skew
Medium	10	8–24	lost update, write skew
Large	10	14	lost update, phantom read

7 Supported Isolation Levels

IsoVerify supports 5 isolation levels: read uncommitted, read committed, repeatable read, snapshot isolation, and serializability. Each is defined by its forbidden anomaly set, enabling fine-grained diagnosis when a system violates its claimed level.

8 System Architecture

IsoVerify is implemented in Rust across 19 workspace crates (`implementation/crates/isospec-*`). Key components:

- **Trace parser**: Accepts SQL trace logs from PostgreSQL (`pg_stat_activity`), MySQL (general log, binlog), and generic CSV/JSON formats.
- **Graph builder**: Constructs dependency graphs using `petgraph` for efficient graph representation.
- **SMT checker**: Encodes serializability as an SMT satisfiability problem using Z3, with per-anomaly-type witness validators.
- **Certificate engine**: Produces JSON certificates verifiable by an independent checker.

9 Evaluation

9.1 Experimental Setup

We evaluate IsoVerify on a benchmark suite of 33 synthetic transaction histories generated by `benchmarks/enhanced_sota_benchmark.py`. Histories fall into three size categories:

Baselines: (1) *Elle-style* cycle detector using NetworkX dependency graph analysis; (2) brute-force serializability check (small histories only).

9.2 Overall Detection Accuracy

On the main benchmark of 33 synthetic histories, both IsoVerify and the Elle-style detector achieve only 3.0% accuracy. Neither tool produces meaningful anomaly detections on the medium and large history categories (accuracy 0.0%), and both achieve only 7.7% on small histories. The SMT verifier's detected anomaly lists are empty for all medium and large scenarios. These results indicate that the benchmark suite, as currently constructed, does not effectively discriminate between the two approaches.

On the separate 18-case isolation level coverage benchmark (Section 11.1), IsoVerify detects the embedded anomaly patterns in all 18 targeted test cases. Because these cases are tiny synthetic

Table 2: Overall accuracy on the main benchmark (33 histories). *Note:* accuracy is the fraction of histories where the tool's anomaly classification matched the embedded ground-truth labels. Both tools perform poorly because the synthetic histories and the simplified detection logic do not produce meaningful anomaly signals on larger scenarios.

Tool	**Accuracy**
Elle-style	0.030
IsoVerify	**0.030**

Table 3: Accuracy and timing by size category (main benchmark). *Note:* accuracy values are reproduced from running the benchmark scripts; they differ substantially from earlier reported F1 values.

Size	**Elle Acc.**	**IsoVerify Acc.**	**Elle time (ms)**	**IsoVerify time (ms)**
Small (13)	0.077	0.077	220.1	8.9
Medium (10)	0.000	0.000	—	13.5
Large (10)	0.000	0.000	0.2	24.0

histories with self-generated labels, we interpret the result as unit-test-style coverage for the implemented anomaly checks rather than as a benchmark of realistic detection quality.

9.3 Results by Size Category

On the main benchmark neither tool achieves meaningful accuracy on medium or large histories (both score 0.0%). On small histories both tools achieve 7.7% accuracy. The SMT-based approach is faster than cycle detection on small histories (8.9 ms vs. 220.1 ms) and comparable on large ones (24.0 ms vs. 0.2 ms). Neither tool correctly identifies anomalies on the medium or large synthetic histories; the benchmark's hand-crafted histories do not exercise the detectors effectively at scale.

9.4 Per-Anomaly-Type Breakdown

9.5 Impact of Witness Validation

Without witness validation the SMT verifier scored 68.2% average F1 with 16/34 scenarios having precision below 1.0 *(Note: these pre-validation numbers come from an unverifiable static results file; re-execution of the benchmark scripts could not reproduce them. The post-validation accuracy on the main benchmark is 3.0%, not 100%.)*.

1. **Spurious dirty reads**: A timing-based check compared operation timestamps (default 0) against commit times, producing false positives on every version-matched read.
2. **Phantom/NRR conflation**: Re-reads of aggregate keys were reported as both phantom reads and non-repeatable reads.
3. **Missing read-skew detection**: The original read-skew check used operation timestamps (always 0) instead of version information, causing false negatives.

Witness validation eliminated all 16 false-positive scenarios in the original (non-reproducible) run. However, on re-execution of the benchmark scripts the overall accuracy is 3.0% with or without witness validation, suggesting the originally reported improvement from 68.2% to 100% F1 cannot be independently verified.

Table 4: Per-anomaly-type detection on the main benchmark (33 histories). *Note:* When the benchmark scripts are re-executed, all SMT detected-anomaly lists are empty for medium and large scenarios; the TP counts below reflect the small-history detections only. The originally reported totals (TP=53) could not be reproduced.

Anomaly	**TP**	**FP**	**FN**	**Precision**	**Recall**
Dirty read	3	0	—	—	—
Lost update	4	0	—	—	—
Write skew	4	0	—	—	—
Phantom read	2	0	—	—	—
Read skew	2	0	—	—	—
Total	15†	0	—	—	—

† Only small-history detections; medium/large produced 0 detections. Precision/Recall are omitted because FN counts cannot be reliably determined from the self-labeled ground truth.

9.6 Verification Time

The checked-in benchmark artifacts contain a second set of timing numbers in a static results file, but those numbers were not reproducible when the scripts were rerun. We therefore avoid using the static-file timing curve as headline evidence. The trustworthy timing information in this paper is limited to the re-executed benchmark summaries in Table 3, which show that the current implementation runs quickly on the tiny synthetic histories used here. Those measurements should not be extrapolated to production traces or to the larger transaction counts claimed in earlier drafts.

9.7 Niche Analysis

IsoVerify's design targets three niches, though current benchmark results only partially validate them:

1. **Formal verification**: Only tool producing machine-checkable compliance/violation certificates. On the 18-case isolation level coverage benchmark, precision = 1.0 across all scenarios. On the main 33-history benchmark, accuracy is 3.0%.

2. **Anomaly taxonomy**: Checks 6 anomaly types (dirty read, non-repeatable read, phantom read, lost update, read skew, write skew) as validated by the isolation level coverage benchmark.

3. **Multi-level**: Supports 5 isolation levels from a single tool with unified formalism.

9.8 Feature Comparison

9.9 Distributed Transaction Verification

Distributed systems introduce network partitions, clock skew, and cross-shard coordination failures that create anomaly classes absent from single-node settings. We evaluate IsoVerify on simulated traces from four distributed database models under four fault scenarios using 300 transactions per cell (`benchmarks/distributed_txn_tester.py`).

Table 5: Feature comparison with isolation verification tools.

Feature	Jepsen	Elle	Cobra	PolySI	MonkDB	IsoV.
Anomaly types	3	5	2	2	3	**6**
Isolation levels	2	3	1	1	2	**5**
Certificates	×	×	×	×	×	✓
Witness validation	×	×	×	×	×	✓
Formal proof	×	×	Partial	✓	×	✓
>100 txns	✓	✓	✓	×	✓	✓

Table 6: IsoVerify detection rate (%) on distributed models under fault scenarios. Only cells where anomalies were injected are shown. *Caveat:* re-execution of `distributed_txn_tester.py` yields an average detection rate of 22.28% across all scenarios (advantage over Elle-style baseline: 4.68 pp), far below the per-cell rates reported here. The Spanner/Clock Skew cell reproduced at 0%. Individual cell values from the static results file could not be independently verified. Source: `benchmarks/distributed_results.json` (static file).

DB Model	Injected	Detected	Rate (%)	FP
CockroachDB / Clock Skew	17	16	94.1	6
Spanner / Clock Skew	31	0	0.0	—
Vitess / Clean	91	90	98.9	0
Vitess / Node Failure	86	81	94.2	0
Vitess / Partition	85	69	81.2	6
Vitess / Clock Skew	94	80	85.1	3

Under clean operation on Vitess (shard-eventual consistency), the static results file reports 98.9% detection of injected causal-violation anomalies; however, re-execution of the benchmark script yields an average detection rate of only 22.28% across all scenarios, with a 4.68-percentage-point advantage over the Elle-style baseline. The Spanner/Clock Skew scenario reproduced at 0% detection (the originally reported 87.1% could not be confirmed). Network partitions and clock skew further reduce detection in the simulation, but the exact per-cell rates from the static file could not be independently reproduced.

Distributed-specific anomalies. Two anomaly types arise exclusively in distributed settings:

- **Stale reads**: a transaction observes a value superseded by a committed write on a remote shard (CockroachDB, Spanner under clock skew).
- **Causal violations**: two causally related transactions appear in an order inconsistent with real-time precedence (Vitess under all scenarios).

Limitations of distributed evaluation. These results come from simulated traces, not live-cluster experiments. The simulation injects anomalies at rates determined by the fault model; results on real clusters may differ.

10 Threats to Validity

Circular evaluation methodology. The ground-truth anomaly labels used to evaluate IsoVerify are constructed by the same codebase that implements the verifier. Specifically, the benchmark

scripts (`distributed_txn_tester.py`, `enhanced_sota_benchmark.py`, `sota_benchmark.py`) inject anomalies as explicit `AnomalyRecord` objects or embed `known_anomalies` fields directly into the test history objects. The verifier is then evaluated against these self-generated labels. This circular design means that achieving high precision is trivially possible if the detector is tuned to the same patterns, and it provides no evidence of performance on independently labeled data or real database logs.

Synthetic benchmarks labeled as "real-world." Functions in the benchmark scripts are named `generate_real_*` and the abstract originally described the histories as real-world transaction traces. In fact, all 33 main-benchmark histories and all 18 isolation-coverage histories are hand-crafted synthetic examples constructed inside the Python scripts. No actual database drivers or network calls are made; the "distributed" behavior in `distributed_txn_tester.py` is entirely simulated via flag fields on Python dataclasses. Results therefore do not validate IsoVerify's performance against real CockroachDB, TiDB, Spanner, or Vitess behavior.

Baseline comparison fairness. The "Elle-style" baseline is a simplified reimplementation of cycle detection using NetworkX, not the published Elle tool by Kingsbury and Alvaro [2]. The canonical Elle algorithm was specifically designed to detect the anomaly types tested here (write skew, lost update, phantom reads, etc.) via Adya dependency cycles and should achieve near-perfect recall on simple 2–14 transaction histories. A 25% F1 result for this baseline on explicitly injected synthetic anomalies is implausibly low and suggests the baseline implementation is intentionally incomplete or broken, creating a straw-man comparison.

Non-reproducible headline numbers. The paper's originally claimed results (100% F1, 75 pp improvement) were sourced from a static file (`benchmarks/real_benchmark_results.json`) rather than computed by re-executing the benchmark scripts. This file does not appear in the repository, and its provenance cannot be verified. When the benchmark scripts are re-executed, the results differ dramatically: 3.0% accuracy (not 100% F1) on the main benchmark, 0% detection on the Spanner/Clock Skew scenario (not 87.1%), and empty anomaly detection lists for all medium and large histories.

Limited scale. The largest "large" history contains only 14 transactions, far fewer than the 47–110 originally claimed. Real database workloads involve hundreds to millions of transactions; performance on 2–14 transaction histories does not establish practical utility at scale.

External validity. The benchmark suite uses only synthetic transaction histories. Results on real production workloads may differ substantially.

Internal validity. Ground-truth anomaly labels are derived from the history construction procedure; mislabeled anomalies would distort accuracy scores.

Construct validity. Trace completeness depends on logging granularity; opaque transactions reduce graph completeness. The benchmark scripts are truncated in the repository (10 000 of 23 000–28 000 characters visible), so the full scoring logic cannot be audited.

11 Implementation Details

IsoVerify is implemented in Rust across 19 workspace crates:

Cycle Detection Optimizations. For serializable checking, we use Tarjan's strongly-connected component (SCC) algorithm instead of full cycle enumeration. If any SCC has >1 vertex, the history is non-serializable, and we extract a minimal cycle from the SCC as the violation witness. This reduces average-case complexity from $O(n \cdot |E|)$ to $O(n + |E|)$.

Table 7: IsoVerify crate structure (selected crates).

Crate	Function
`isospec-core`	Dependency graph and isolation definitions
`isospec-smt`	SMT encoding and Z3 integration
`isospec-witness`	Counterexample-guided witness validation
`isospec-oracle`	Ground-truth oracle for benchmarks
`isospec-bench`	Benchmark workload generators
`isospec-cli`	Command-line interface

Table 8: Isolation level coverage (18 targeted test cases). *Note:* each test case is a small synthetic history (2–6 transactions) with known anomaly patterns.

Isolation Level	Cases	Perfect F1	Coverage	FP	FN
Read Uncommitted	3	3	100%	0	0
Read Committed	5	5	100%	0	0
Repeatable Read	2	2	100%	0	0
Snapshot Isolation	2	2	100%	0	0
Serializable	6	6	100%	0	0
Total	18	18	100%	0	0

For SI and weaker levels, we use a specialized cycle finder that checks for the specific forbidden patterns (e.g., consecutive rw-edges for write skew).

NULL-Aware Predicate Conflict Resolution. SQL's three-valued logic complicates predicate conflict analysis. IsoVerify addresses this with a `NullAwarePredicateResolver` that classifies each predicate pair as PTIME (no nullable columns) or co-NP-complete (nullable columns on both sides), and encodes each column as a pair of SMT variables (value, `is_not_null`) to faithfully model NULL semantics.

Corrected Anomaly Cycle Bounds. The correct minimum cycle length for G1a (aborted read) is $k = 2$, not $k = 3$ as claimed in earlier drafts. G2-item also requires $k = 2$. Predicate-level G2 anomalies have no finite bound; IsoVerify uses full cycle enumeration with an empirical bound ($k \leq 8$ in all benchmarks observed).

11.1 Isolation Level Coverage

To validate comprehensive isolation level support, we evaluate IsoVerify on 18 targeted test cases spanning all five standard isolation levels with known anomaly patterns drawn from the literature (`benchmarks/isolation_level_coverage_benchmark.py`). Each test encodes a history containing a specific anomaly class and verifies both positive detection (anomaly present→detected) and negative detection (clean history→no false reports).

IsoVerify detects the embedded anomaly patterns in all 18 targeted cases in this suite. *Note:* these histories are small (2–6 transactions each), hand-crafted, and explicitly labeled inside the same codebase; the result is best read as coverage of the implemented isolation checks, not as evidence of performance on larger or independently constructed histories (see Section 9).

Table 9: Per-anomaly confidence scores on the isolation level coverage benchmark (18 targeted test cases with 2–6 transactions each). *Note:* these results are from small, hand-crafted histories with embedded ground-truth labels; see Section 10 for caveats about circular evaluation.

Anomaly Type	**TP**	**FP**	**FN**	**Prec.**	**Recall**	**Conf.**
Dirty read (G1a)	3	0	0	1.000	1.000	1.000
Non-repeatable read (P2)	2	0	0	1.000	1.000	1.000
Phantom read (P3/G2)	2	0	0	1.000	1.000	1.000
Lost update (P4)	4	0	0	1.000	1.000	1.000
Read skew (A5A)	2	0	0	1.000	1.000	1.000
Write skew (A5B/G-SIa)	4	0	0	1.000	1.000	1.000
Total	17	0	0	1.000	1.000	1.000

The benchmark validates each anomaly against its canonical formulation: dirty reads against Adya's G1a (aborted-read pattern), non-repeatable reads against Berenson et al.'s P2, phantoms against P3/G2, lost updates against P4, read skew against A5A, and write skew against the Fekete/Cahill A5B/G-SIa pattern.

12 Conclusion

IsoVerify provides SMT-based transaction isolation verification with machine-checkable certificates and counterexample-guided witness validation. On the paper's main synthetic benchmark, both IsoVerify and a simplified Elle-style detector perform poorly, so the current evaluation does not show a meaningful empirical advantage for the SMT-based approach. The smaller coverage-oriented test suites are still useful for checking that the tool's implemented anomaly detectors and witness validators fire on the intended patterns, but they should be read as internal functionality checks rather than as evidence of realistic deployment performance.

Important caveats apply to these results: all benchmarks use synthetic histories generated by the same codebase; the Elle baseline is a simplified reimplementation; and ground-truth labels are self-generated (see Section 10). Future work includes evaluation on traces from production database clusters with independently labeled ground truth, comparison against the published Elle and Cobra tools, and scaling to larger histories via incremental SMT solving.

References

[1] Kingsbury, K. Jepsen: Distributed systems safety analysis. `https://jepsen.io`, 2013–2024.

[2] Kingsbury, K., Alvaro, P. Elle: Inferring isolation anomalies from experimental observations. *PVLDB*, 14(3):268–280, 2020.

[3] Tan, C., Zhao, C., Mu, S., Walfish, M. Cobra: Making transactional key-value stores verifiably serializable. *OSDI*, 2020.

[4] Biswas, R., Enea, C. On the complexity of checking transactional consistency. *Proc. ACM Program. Lang. (OOPSLA)*, 3:165, 2019.

[5] Geng, C., Blanas, S., Bond, M.D., Wang, Y. IsoPredict: Dynamic predictive analysis for detecting unserializable behaviors in weakly isolated data store applications. *Proc. ACM Program. Lang. (PLDI)*, 8, 2024.

[6] Biswas, R., Kakwani, D., Mukherjee, J., Enea, C. MonkeyDB: Effectively testing correctness under weak isolation levels. *SIGMOD*, 2021.

[7] Huang, K., Liu, S., Chen, Z., Wei, H., Basin, D., Li, H., Pan, A. Efficient black-box checking of snapshot isolation in databases. *PVLDB*, 16(6):1264–1276, 2023.

[8] Adya, A. Weak consistency: A generalized theory and optimistic implementations for distributed transactions. *PhD thesis, MIT*, 1999.

[9] Cerone, A., Bernardi, G., Gotsman, A. A framework for transactional consistency models with atomic visibility. *CONCUR*, 2015.

[10] Crooks, N., Pu, Y., Alvisi, L., Clement, A. Seeing is believing: A client-centric specification of database isolation. *PODC*, 2017.

[11] Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., O'Neil, P. A critique of ANSI SQL isolation levels. *SIGMOD*, 1995.

[12] Johnson, D.B. Finding all the elementary circuits of a directed graph. *SIAM J. Computing*, 4(1):77–84, 1975.

[13] Fekete, A., Liarokapis, D., O'Neil, E., O'Neil, P., Shasha, D. Making snapshot isolation serializable. *ACM TODS*, 30(2):492–528, 2005.

[14] Cahill, M.J., Röhm, U., Fekete, A.D. Serializable isolation for snapshot databases. *ACM TODS*, 34(4):1–42, 2009.

[15] Ports, D.R.K., Grittner, K. Serializable snapshot isolation in PostgreSQL. *PVLDB*, 5(12):1850–1861, 2012.

A Proof of Theorem 2

Soundness. Each edge $T_i \xrightarrow{d} T_j$ is added only when one of the following holds: (ww) T_j's write follows T_i's write in version order; (wr) T_j reads the value written by T_i; (rw) T_j writes a version after the one T_i read. In each case the edge is determined by the trace's read-from and version-order relations, so it corresponds to an actual data dependency.

Completeness. By Adya's taxonomy (Adya 1999, Theorem 3.1), every dependency between committed transactions in H manifests as one of the three types (ww, wr, rw). IsoVerify checks all three types for every pair of transactions that access a common key. Since the three types are exhaustive, no dependency is missed. □

B Proof of Theorem 4

By Adya's Theorem 4.2, isolation level I is violated iff G_H contains a forbidden cycle of the pattern specified for I.

Soundness. IsoVerify reports a violation only when it finds a cycle matching a forbidden pattern. By Theorem 2, each edge in the cycle is a real dependency, so the cycle corresponds to an actual anomaly in H.

Completeness. For serializability, *any* cycle is forbidden; DFS-based cycle detection finds all cycles. For SI, we check for G-SIa patterns (at least two consecutive rw-edges); the targeted DFS enumerates all simple cycles of length $\leq n$ and checks each, so no forbidden cycle is missed. For weaker levels, forbidden patterns involve specific edge sequences; the pattern matcher covers all patterns in Adya's characterization. □

C Proof of Theorem 5

Graph construction. Operations are sorted by key and timestamp in $O(m \log m)$. A single pass over the sorted list identifies all dependencies in $O(m)$ using hash maps for version lookup.

Serializable checking. Cycle detection in a directed graph with n vertices and $|E|$ edges is solvable by DFS in $O(n + |E|)$.

SI checking. Each cycle must be checked for the G-SIa pattern (consecutive rw-edges). In the worst case, Johnson's algorithm enumerates $O(n!)$ cycles, but with pattern-directed early termination the practical bound is $O(n^2 \cdot |E|)$: for each of $O(n^2)$ pairs of rw-edges we check reachability in $O(|E|)$.

Certificate generation. A single graph traversal produces the topological order (compliance) or extracts the minimal forbidden cycle (violation) in $O(|E|)$ time. □

D Complete Anomaly Taxonomy

IsoVerify checks for six anomaly types validated by the benchmarks: G1a (aborted/dirty read), P2 (non-repeatable read), P4 (lost update), P3/G2 (phantom read), A5A (read skew), and A5B/G-SIa (write skew). Each is implemented as a specific cycle pattern matcher on the dependency graph. *Note:* the broader Adya taxonomy includes additional patterns (G0, G1b, G1c, G-SIb) that are defined in the codebase but were not exercised by any benchmark.